

Auth, Tenant Isolation & API Security Foundation

Confidential | March 2026 | GalaxyBots.ai

Auth, Tenant Isolation & API Security Foundation

What & Why

Every API endpoint on GalaxyBots is currently open — there is no authentication or authorization layer, meaning any client's data is accessible to anyone who knows the URL. Before the platform can serve real enterprise clients, it needs a mandatory auth layer (JWT-based sessions), strict per-client/tenant data isolation, rate limiting on expensive AI endpoints, and correct CORS configuration.

This is the single highest-priority gap identified by the platform audit. All other features (billing, white-label, BingoLingo integration) depend on this foundation being in place.

Done looks like

- All API routes require a valid session token (JWT or signed cookie) to access; unauthenticated requests receive `401`
- Users can register and log in via the GalaxyBots frontend; a session token is issued and stored client-side
- Every database query is scoped to the authenticated user's `clientId` — no cross-tenant data access is possible
- Role-based access: at minimum, `owner`, `admin`, and `viewer` roles exist and are enforced on sensitive routes (e.g., only `owner` / `admin` can trigger task sessions or manage integrations)
- Bot memory and session context in `services/memory.ts` is strictly scoped by `clientId` so no data bleeds between tenants
- Rate limiting is applied to all LLM/OpenAI paths (`/api/task-sessions`, `/api/bots/generate-declarations`, `/api/boardroom/messages`) — configurable per plan tier
- CORS is restricted to known origins (not permissive wildcard) in production
- The scheduler service uses a distributed lock or single-leader pattern to prevent duplicate job execution on horizontal scale
- A basic audit log table records key actions (session start, tool execution, integration connected) per client for compliance readiness

Out of scope

- Full SSO/SAML/OIDC enterprise identity (can be added later once RBAC is in place)
- Per-user fine-grained permissions beyond owner/admin/viewer

- Billing enforcement (covered in Task #8)
- Frontend redesign — auth forms should integrate cleanly into the existing UI

Tasks

1. **Auth infrastructure** — Implement JWT-based authentication using a standard library (e.g., `jsonwebtoken` + `express-jwt`). Add `/api/auth/register`, `/api/auth/login`, `/api/auth/logout`, and `/api/auth/me` endpoints. Store hashed passwords and user records in a new `users` table linked to `clientId`.
1. **Auth middleware enforcement** — Create an `authenticate` middleware and apply it globally to all protected routes in `app.ts`. Attach `req.user` (`userId`, `clientId`, `role`) on every request that passes auth.
1. **Tenant-scoped query guards** — Audit every route in `routes/` and enforce that all DB queries filter by the authenticated `clientId`. Refactor `services/memory.ts` to scope all memory reads/writes by `clientId`.
1. **RBAC roles** — Add an `role` field to the `users` table (`owner`, `admin`, `viewer`). Enforce role checks on destructive or expensive routes (task session creation, integration management, bot fabrication).
1. **Rate limiting** — Apply per-client/per-IP rate limits to all LLM-backed routes using a library like `express-rate-limit`. Limits should vary by plan tier (stored on the client record) and return `429` with a `Retry-After` header.
1. **CORS hardening** — Restrict allowed origins in `app.ts` to the known frontend domains (GalaxyBots, BingoLingo) rather than wildcard. Use environment variables for domain configuration.
1. **Scheduler distributed lock** — Add a simple advisory lock (PostgreSQL `pg_try_advisory_lock`) around the scheduler startup so only one instance runs scheduled jobs in multi-server deployments.
1. **Audit log** — Add a `platform_audit_log` table and write a lightweight logging middleware that records key events (login, task session start, tool execution, integration connect/disconnect) per client, queryable via a new `GET /api/audit` endpoint.
1. **Frontend auth flow** — Add login and registration pages to the GalaxyBots frontend. Redirect unauthenticated users to login. Store the JWT in `httpOnly` cookies or secure `localStorage`. Wire the existing API client to include the auth header on all requests.

Relevant files

- `artifacts/api-server/src/app.ts`
- `artifacts/api-server/src/index.ts`
- `artifacts/api-server/src/routes/index.ts`
- `artifacts/api-server/src/routes/clients.ts`
- `artifacts/api-server/src/routes/task-sessions.ts`
- `artifacts/api-server/src/routes/client-integrations.ts`
- `artifacts/api-server/src/services/memory.ts`
- `artifacts/api-server/src/services/scheduler.ts`
- `artifacts/api-server/src/tools/agent-ic-loop.ts`
- `lib/db/src/schema`

